Part 4C — James Cabreros (11:00 slot; schedule listed Zhaowen Lan) — Stories C

Scope: release governance and legacy-engine migration (design cases on lived record), AI-assisted configuration tooling, policy-platform sequencing, LP router, questions to ask

Friday, October 30, 2026 · Amazon L6 Senior Supply Chain Manager, NACF Supply Chain · Rajesh Rishiyur Chandrasekaran

Contents

1. J5 · E8 — Monthly release / regression governance (DESIGN CASE) on the lived S15 four-week cadence DESIGN CASE
2. J7 · E9 — Migrating a legacy rules engine without unintended deployments (DESIGN CASE) on the lived S18 RP→JDA cutover DESIGN CASE
3. AI-assisted configuration tooling — one page (I1: guardrails and operator self-service angle) NEW
4. Legacy rules engine → policy platform: how I would sequence it — one page DESIGN
5. Router for James — 16 LPs and process prompts → exact story pointers (real AIQB wordings)
6. Questions to ask James · source trace · VERIFY list with safe sentences

Rules for this part. E8 and E9 are **DESIGN CASES** from my interview-prep design engagement — I say "I designed / here is how I would run it," never "I delivered." Every design case is paired with a lived program I can be probed on (S15, S18, UDC B3, A4, I1). Basis labels: **record** (resume / Apple v11 / master canon) · **record-from-docs** (a page of a source document) · **design** (my proposed mechanism) · **illustrative**. Angle rule: Rocky hears the S18 and UDC programs as program/cutover; James hears them as process, configuration governance and validation. Already told Thursday (never primary here): S/4HANA / ISM, Flexport estimation, Shipment Visibility, safety stock, the 200-lane program.

1. J5 · E8 — Release and regression governance: how a change earns its way to production DESIGN CASE

DESIGN CASE + LIVED RECORD. The monthly SAP + integrations release train (E8) is a design I wrote in an interview-prep engagement; volumes illustrative. The record under it is lived: **S15** — the Sephora 20-person Product/Engineering/QA team moved from a six-week hybrid cadence to four weeks in six weeks, QA authored the release gate, critical defects −50% (record: resume; master S15). Sephora UDC facts used here are record: hard peak change-freeze October–December; anything touching the frozen spec went through the gate with an estimate; hypercare exited into the monthly release train (record: Apple v11 B1 p. 21–22; GFiber v8 S2).

His bullet it answers: "configuration governance — pre-deployment validation, change-management SOPs, post-incident corrective actions; fewer unintended deployments; auditability across hundreds of weekly changes."
LPs: Insist on the Highest Standards · Hire and Develop the Best · Earn Trust · Invent and Simplify |
Headline: four-week cadence in six weeks; critical defects per release −50% (record).

60-SECOND PITCH

"At Sephora the VP asked me to move a 20-person Product, Engineering and QA team from a six-week hybrid cadence to four weeks. QA feared compressed test windows, and they were right. I had no line authority, so I made the skeptic the owner: the QA lead wrote the definition of done, I coached the frame, and we added a QA drop date with reason codes so late, heavy stories were dropped on the record instead of squeezing the test window. Four-week cadence live in six weeks; critical defects per release down 50%; neighbouring teams copied the model. Later I designed the same mechanism as a monthly release train for SAP plus integrations: scope frozen at T-minus-15, regression cells owned at the true grain, a go/no-go the QA lead owns with blocker-plus-critical at zero, rollback per transport group, and hypercare that exits on a number. The principle for your world is the same: a change rides the gate or it does not ship, and every emergency change is counted."

S — Situation. *Lived (S15, record).* Newly joined at Sephora, the VP of Supply Chain asked me to convert a 20-person Product/Engineering/QA team from a six-week hybrid cadence to four-week agile. QA (a contracting firm) and in-house engineering were heading for a standoff: QA feared compressed test windows, and the Sev-1 trend was partly from story-accuracy defects. *Design case (E8, design).* The prompt: SAP configuration, custom objects, integration maps, the CRM, the 3PL interface and the field app all change; last month a release broke MRP. Design a train so changes ship monthly with no surprise on the floor.

T — Task. Convert the cadence in weeks, not quarters, with VP air cover and no line authority over anyone in the room — and keep escaped criticals from rising. In the design: one book of record for change (the transport/change log and the release manifest) and one primary metric — release-caused Sev 1–2 incidents per release driven to zero with the cadence held.

A — Action. What I did (lived), then what I designed on top of it.

- **Discovery (lived).** One-on-ones with every person; an influencer per function; the problem statement written as "QA's fear of compressed windows is correct." Exit: every real objection heard privately; the QA lead agreed to author the gate.
- **Gate ownership (lived).** The QA lead wrote the definition of done — entry criteria per story type at the Ready-for-QA transition. Engineering got review, not veto, and shaped the entry criteria. I wrote the success criteria with the VP: four-week cadence, no rise in escaped criticals, QA window protected by rule.
- **Risk-scoped intake (lived).** Jira fields for benefit ranking and review status; a QA drop date with reason codes so a late heavy story was re-scoped on the record; QA seated on product-business calls with pre-approved scenarios; Sev-1 causes classified.
- **Freeze and change control (lived, Sephora UDC).** Hard peak change-freeze October–December; anything touching the frozen spec went through the gate with an estimate, never absorbed quietly; after hypercare the design was handed into the monthly release train.
- **The train (design, E8).** Commit and freeze at T−15 (definition of ready cites a regression cell) → promote to integrated QA at T−10, imported in production sequence → regression matrix plus end-to-end chain at T−5 → go/no-go at T−2, QA lead owns it: Blocker + Critical open = 0, rollback plan per transport group, freeze check → deploy → hypercare with a numeric exit. Release notes and the go/no-go record are the audit artifacts.
- **Regression scope by risk (design).** Cells at the true grain (order type × fulfillment path × node × stock-state transition; interface round-trips per partner); an MRP-protection cell that diffs exception counts and planned-order totals by plant before vs. after the transport — an unexplained delta fails the build. Build the manual matrix first; automate the cells that fail most.
- **Corrective action (design).** Emergency changes are allowed, counted in the postmortem and 100% retrofitted into the train; configuration and code ride separate transports so a config rollback never drags code.

R — Result.

- Four-week cadence live in six weeks for a 20-person Product/Engineering/QA team `record`.

- Critical defects per release −50% under the drop-date rule record ; the QA lead defended the gate to engineering without me; neighbouring teams copied the ceremony from the user guide record .
- Rollback (revert to six weeks) named at kickoff as a two-way door; never used record .
- E8 target design metric: release-caused Sev 1–2 per release → 0, twelve trains a year with no skipped month, emergency changes ≤ 1 per release and 100% retrofitted design — a target, not a result.

TRADE-OFF I MADE. Options: keep six weeks; go to four weeks and absorb late stories; or go to four weeks with a drop-date rule. I chose the rule and dropped low-impact stories on the record — engineering's date protected, QA's window protected — and covered some of the drops myself. Cost: visible scope loss each cycle. In the design, a four-week train delays small fixes; I accept that and keep a counted emergency path rather than let urgency become a side door.

FAILURE / LEARNING. Development that depends on me staying is not development: the gate became durable only when its owner, not I, defended it. Hard feedback that I routed sideways through issues-log rows turned out partly to be genuine overload, and the evidence got those people help — I now check load before I treat pushback as resistance record: master S15 probes .

Insist on the Highest Standards — the gate could say no and survive a release-day argument; criticals −50%.
Hire and Develop the Best — I handed the load-bearing artifact to the skeptic and coached her to own it.
Earn Trust — I said publicly that QA's fear was correct before asking for a faster cadence.
Invent and Simplify — a drop date with reason codes replaced a last-two-days bug bash.

SAFE-SAY. "The six-week conversion and the 50% cut in critical defects are my record at Sephora; the dated monthly train with T-minus-15 and the MRP diff cell is a design I wrote from that experience, not something I claim to have run at that scale." **If asked for source:** "My resume line on the VP-sponsored conversion, and the master story record S15; the train is design case E8."

20-SECOND VERSION

"I moved a 20-person Product, Engineering and QA team to a four-week cadence in six weeks by making the QA lead the owner of the release gate and adding a drop date with reason codes. Critical defects fell 50%. I later designed the same gate as a monthly SAP release train: frozen scope, risk-scoped regression cells, a numeric go/no-go and counted emergency changes."

PROBES JAMES IS LIKELY TO ASK

1. You run hundreds of configuration changes a week, not a monthly train. What carries over?

The gate, not the calendar. I would classify changes by blast radius: low-risk rows go self-serve with automated validation and an audit row; high-risk rows (new node types, escape routes, capacity signals) ride a scheduled window with a go/no-go and a named rollback. The invariant is the same as at Sephora: nothing deploys that is not on the manifest, and every exception is counted. design

2. How did you scope regression by risk rather than testing everything?

Lived: stories were benefit-ranked in Jira and the QA lead's entry criteria differed by story type; heavy late stories got a drop date. Design: the matrix is at the grain the failure happens — node × channel × transition — and the cells that failed most get automated first.

3. What happens when a release still breaks production?

Roll back the transport group, not the whole release; open a corrective action with a root-cause class; add the missing regression cell so the same class cannot pass again. In the design, the MRP diff cell exists because a release broke MRP. The cell, not the apology, is the corrective action.

4. How did you get engineering to accept a gate written by QA?

Review, not veto: engineering shaped the entry criteria, and I covered some of the low-impact stories they had to drop. Objection became co-ownership. Two cycles on four weeks, and the QA lead was defending the gate herself.

2. J7 · E9 — Migrating a legacy rules engine without unintended deployments: tolerance first, mocks by cause, one publisher DESIGN CASE

DESIGN CASE + LIVED RECORD. E9 (the ERP cutover from a legacy WMS) is a design I wrote in an interview-prep engagement; volumes illustrative. The record under it is lived: **S18** — Sephora RP → JDA WMS migration across two DCs in eight months: a 0.5% reconciliation tolerance published with Finance before mock 1; three mocks at 7% → 1.8% → 0.3%; a 72-hour in-transit freeze; legacy read-only as failback; a $20M ROI pipeline of four programs unblocked (record: resume; master S18). The “one publisher” rule is lived on Sephora UDC: no SKU with two published availability values (user anchor: GFiber v8 S2). **Angle for James:** the governance mechanics — parity, mapping as data, validation before volume, one source of truth per node. Rocky hears S18 as a program and cutover story; do not retell the budget or the team here.

His bullet it answers: “leads migration of a legacy placement rules engine to a next-generation centralized policy platform … architecture decisions, stakeholder alignment, sequencing, training delivery” and “fewer unintended deployments.”
LPs: Insist on the Highest Standards · Dive Deep · Learn and Be Curious · Bias for Action | **Headline:** 7% → 1.8% → 0.3% against a 0.5% tolerance published before the first mock (record).

60-SECOND PITCH

“The closest thing I have run to your rules-engine migration is a WMS migration at Sephora: two DCs on a legacy system with no documentation and an eight-month deadline, and I had never run a WMS migration. The risk was the same as yours — the new platform behaves differently from the old one in a way nobody notices until it ships. So I made ‘good enough’ arithmetic before anyone tested: a 0.5% reconciliation tolerance published with Finance before the first mock. I treated mappings — units of measure, location names, status vocabularies — as data in governed tables, not code. Each mock was read by cause class, never by the headline: 7% was mapping, 1.8% was in-transit timing, 0.3% passed. At cutover a 72-hour freeze, the legacy system read-only as failback, and one rule that I designed from it: zero the old publisher before the new one publishes, so consumers read one source at a time. When hundreds of RF devices failed the conversion script, the rehearsed paper-pick fallback fired instead of a rollback; training moved 1.5 weeks and the date held.”

S — Situation. *Lived (S18, record).* Two Sephora DCs ran a legacy WMS with zero inherited documentation; a hard eight-month deadline gated a $20M ROI pipeline of four programs; inventory accuracy across two DCs could not wobble at cutover. I had never run a WMS migration. *Design case (E9, design).* A site runs a legacy system with its own book; the business wants the new platform to become the book of record with no double-promising during the switch and a Finance-grade reconciliation at go-live.

T — Task. Own the lifecycle — charter, design sign-off, interface inventory, UAT, cutover, hypercare — accountable to the sponsor and Finance, with one test of success that nobody could negotiate after the fact. Translated to a rules-engine migration: prove the new platform produces the same decisions as the old one, within a published tolerance, before it decides anything live.

A — Action. Five gates; each step is lived unless marked design.

- **Discovery — inventory before opinions.** Charter written in week one with the success criteria (on the date, inside a published tolerance, zero accuracy incidents, hypercare exit under two blockers a day). Interface inventory v0 built from message logs; every interface got a named owner and a test cell. I learned the floor in month one until I could walk receive → put-away → pick → ship unaided.
- **Design — tolerance and mapping as data.** The 0.5% tolerance published with Finance before mock 1. Conversion order fixed: masters → open orders → on-hand → in-transit as its own batch. A mapping-table standard: units of measure, locations and status vocabularies as governed data, not code. Stories slotted by business criticality × effort.

- **Build & Test — mocks read by cause.** Three mocks on production data with written root causes. Mock 1 at 7%: units of measure, location names that mapped to nothing, order-status vocabulary. Fixed in the mapping set → 1.8%. In-transit timing → the 72-hour freeze and a separate batch → 0.3%. A total that shrank without a named cause was not evidence.
- **Cutover — one door, one publisher, rehearsed fallbacks.** Written go recommendation off mock 3; the 72-hour in-transit freeze; the legacy system read-only as failback; rollback triggers per interface written in advance (mismatch above 0.5% for two days, or ship-confirm posting failing beyond two hours) — rehearsed, never used. Hundreds of RF devices failed the conversion script; I invoked the rehearsed paper-pick fallback rather than roll back and told leadership training would slip 1.5 weeks. *Design (E9):* zero the legacy availability first, then publish the new — never the reverse.
- **Hypercare — numeric exit.** Blockers per day read daily; exit at under two a day sustained, in week 3 of 4; the exit memo carried the single-point-dependency lesson.

R — Result.

- Reconciliation variance 7% → 1.8% → 0.3% against the 0.5% tolerance published before mock 1 record.
- Two DCs live on JDA in eight months, on time; the $20M ROI pipeline of four programs unblocked the next month — slip avoided, not banked record.
- Per-interface rollback rehearsed and never needed; the paper-pick fallback fired for the RF miss; training moved 1.5 weeks and the go-live date held record.
- Hypercare exited in week 3 at under two blockers a day record. E9 target metric: cutover variance inside the published tolerance with zero double-promises design.

TRADE-OFF I MADE. Options: one mock and a long hypercare, or three full mocks. I chose three — they cost weeks that would otherwise have been spent live. I also chose the 72-hour freeze, which delayed receipts, priced with operations and scheduled off-peak. At the RF failure I chose fallback over rollback: a rollback would have reopened every converted interface to save one floor task.

FAILURE / LEARNING. The RF miss was mine: RF conversion sat on the log as a task, not a risk, because I scored it by effort when impact mattered. Device testing now precedes training, and every single-point dependency gets a what-if plan regardless of size record.

Insist on the Highest Standards — the tolerance was published before I knew I could hit it.
Dive Deep — every mock delta carried a root-cause class and an owner.
Learn and Be Curious — I had never run a WMS migration; I learned the floor in month one.
Bias for Action — the rehearsed fallback fired in minutes instead of a rollback debate.

SAFE-SAY. “Hundreds of RF devices failed the conversion script; we ran the rehearsed paper fallback, training moved a week and a half, and the go-live date held.” Never state a device count. **If asked for source:** “My resume line on the dual-DC migration — three mocks to a published 0.5% tolerance — and the master record S18; the rules-engine framing is design case E9.”

20-SECOND VERSION

“I migrated two DCs off a legacy WMS in eight months by publishing a 0.5% reconciliation tolerance with Finance before the first mock, treating mappings as governed data, and reading every mock by cause: 7%, 1.8%, then 0.3%. At cutover one source published at a time and the fallbacks were rehearsed — one fired, and the date held.”

PROBES JAMES IS LIKELY TO ASK

1. How would that translate to a placement rules engine with no inventory to reconcile?

The “variance” becomes decision parity: replay a frozen set of real placement requests through both engines and diff the decisions by cause class — rule missing, rule translated differently, data mapping, intended change. The tolerance is published before the first replay; intended changes are listed in advance so they are not counted as defects. design

2. How do you avoid two engines deciding at once?

One publisher per node at a time: the legacy engine stops deciding for a node before the platform starts, never the reverse, with the switch logged. On UDC the rule was no SKU with two published availability values; here it would be no node with two active rule sources.

3. What were your rollback triggers?

Written in the charter before go-live: mismatch above tolerance for two days, or posting failing beyond two hours, with rollback per interface and the legacy system read-only as the failback. Rehearsed; never used. Per-interface rollback is why one bad interface never forced a full rollback.

4. What would you do differently?

Score dependencies by impact, not effort. The RF conversion looked small, so it was a task rather than a risk. Now any dependency that can stop the floor gets a rehearsed fallback and a test before training.

3. AI-assisted configuration tooling — operator self-service with guardrails ONE PAGE

Lived analog and angle. I1 / Apple A01 — an LLM incident-triage agent I built independently (Python/FastAPI), **not an employer production deployment** (record: Apple v11 I1, pp. 41–42). Venkata may hear I1 as a product story; James hears only the **guardrails, validation and operator self-service** angle.

THE ANSWER (ABOUT 60 SECONDS)

"I built an independent LLM triage agent as an operations product, not a demo. Three guardrails made it safe. First, the eval set came before any prompt or model change: I graded historical incidents with known-correct outcomes, and every change had to keep the past passing — a pre-deployment validation gate. Second, 'no citation, no proposal': every recommendation had to cite the ticket evidence and the runbook line. Third, tiered routing: easy, well-understood classes went to small models; the expensive path was reserved for the hard tail. Cost went from $25 to $0.40 per incident, a 98% cut, with accuracy held on the eval set and 100% citation coverage. For configuration tooling I would keep those three and add one: the tool proposes, a validator decides, and the operator approves a readable diff before anything deploys."

S — The naive agent cost $25 per incident and could answer without checkable evidence, so the human repeated the work. **T —** Quality held on known cases, every proposal checkable, unit cost below a dollar. record

A — Eval set graded first and made the release gate; "no citation, no proposal"; cost measured per class and routine classes routed to smaller models; accuracy, citation coverage and unit cost reviewed together. record

R — $25 → $0.40 per triaged incident (98% reduction); accuracy held on the eval set; 100% citation coverage. record

GUARDRAIL I BUILT (I1, RECORD)	CONFIGURATION-TOOLING EQUIVALENT (DESIGN)	LIVED EVIDENCE THAT THE PATTERN WORKS AT NETWORK SCALE
Eval set of graded historical cases; every change must keep the past passing	A regression pack of past network changes and known-good outcomes; a proposed change is replayed against it before deploy	S18 mocks read by cause against a published tolerance; E8 MRP diff cell record / design
No citation, no proposal	Every generated change cites the rule, SOP step and data row it relies on; uncited changes cannot be submitted	UDC 44-row register: one change, one system, one owner per row record-from-docs
Tiered routing — easy classes to small models, hard tail reserved	Low-blast-radius change classes self-serve; new node types, escape routes and capacity signals route to a specialist review	Canada node enablement: six changes, five small, run by a seven-step script before the nightly batch record-from-docs
Accuracy, citations and cost read together	Weekly metrics: changes self-served, validation-fail rate, rollbacks, unintended deployments, review latency	S15 drop-date reason codes made late work visible record
Human approval kept until evidence beats the human baseline	Operator approves a readable before/after diff; auto-approve only for classes with a clean record	A5 launch gate: shadow accounts on named customers, 80% minor-to-no-issues bar before customer-facing use record

TRADE-OFF I MADE. One expensive model for every case was simpler; tiered routing added routing logic and eval upkeep but kept quality and made the economics viable. For operators: more change classes to govern, in exchange for self-service that is safe by default.

FAILURE / LEARNING. My early eval set was still growing while prompts were being judged, so early comparisons were not like-for-like. I would freeze and version the baseline before the first experiment — for configuration tooling, version the regression pack with every rule release. record

LPs: Invent and Simplify (self-service with guardrails built in) · Insist on the Highest Standards (no citation, no proposal) · Frugality ($25 → $0.40) · Dive Deep (graded history before changes).

SAFE-SAY. "This was my independent build, not an employer production program: $25 to $0.40 per triaged incident, quality held on my graded eval set, and every proposal cited." **Source:** "My build records, captured in Apple workbook I1, pages 41–42." **20 seconds:** "I built an incident-triage agent with the eval set first and one rule — no citation, no proposal. Cost fell from $25 to $0.40 per incident with quality held. For configuration tooling: the tool proposes, a validator decides, the operator approves the diff."

Probe: what breaks at Amazon scale?

Access review, data boundaries, regression-pack upkeep and drift become programs of their own; keep human approval until production evidence beats the human baseline. design

4. Legacy rules engine → policy platform: how I would sequence it DESIGN ANSWER

DESIGN ANSWER ON LIVED EVIDENCE. I have not migrated a placement rules engine. Each step below is my proposed sequence (**design**), and each is backed by a mechanism I ran (**record / record-from-docs**). UDC rows: same Sephora UDC umbrella Rocky may have heard; governance facet only.

THE ANSWER (ABOUT 45 SECONDS)

"I would sequence it in seven steps, and none of them deploys anything until parity is proven. Inventory every legacy rule as a row with an owner and a test cell. Publish the parity tolerance before the first replay, so 'good enough' is arithmetic. Run the platform in shadow, deciding alongside the legacy engine but executing nothing, and read every disagreement by cause. Cut over in waves by node type, with one rule source per node at a time. Then every change rides pre-deployment validation, rollback triggers are written in advance, and legacy retires only when every consumer has a destination."

STEP	WHAT I WOULD DO (DESIGN)	EXIT GATE	LIVED EVIDENCE
1. Parity matrix	Every legacy rule as a row: rule, node types touched, owner, test cell, and an "intended change" flag so planned differences are not counted as defects	Every rule owned; intended deltas signed	UDC: 44 numbered rows across 23 systems, one change and one system per row (council deck pp. 13–14) record-from-docs ; S18 interface inventory from message logs, owner + test cell each record
2. Tolerance, then replay	Publish the decision-parity tolerance with Operations and Finance; replay a frozen set of real placement requests through both engines; diff by cause class	Replay inside tolerance, every delta classed	S18: 0.5% published before mock 1; 7% → 1.8% → 0.3%, each mock removing a named cause record
3. Shadow mode	Platform decides alongside legacy, logged not executed, on live volume; disagreements triaged daily	Disagreement rate flat at or below tolerance for an agreed run	Flexport launch gate: shadow accounts on named customers, 80% minor-to-no-issues bar before customer-facing use (A5) record ; UDC interim bridge: planning auto-copied 801 → 900 so it saw one node before the floor did (rows 19, 21) record-from-docs
4. Waves by node type	Simplest node type first, then crossdock, then new node types; one rule source per node at a time — legacy stops deciding before the platform starts, switch logged	Wave exit on numbers, not dates	Canada: 99 brands in waves of 20 / 40 / 39 (discovery p. 16) record-from-docs ; UDC: no SKU with two published availability values user anchor
5. Pre-deployment validation	After cutover every change rides a manifest: replay against the regression pack, readable diff, approval by change class, audit row	Unintended deployments trending to zero	Canada: five business data sets, seven IT steps before the nightly planning batch (p. 19) record-from-docs ; E8 train design ; S15 QA-owned gate record
6. Rollback	Per node and per rule set, triggers written before go-live; legacy kept read-only through the failback window	Rehearsed in the last mock	S18: rollback per interface, triggers written in advance, legacy read-only as failback; rehearsed, never used record
7. SOPs, training, decommission	Runbook per change class; train on real changes once tooling is proven; retire legacy only when every consumer has a destination	Zero orphan consumers	A4: all 79 legacy Crux customers ($185,892 ARR) given migrate / white-label / cancel paths before retirement record-from-docs ; S18 RF lesson — device test before training record ; S15 user guide copied by neighbouring teams record

Weekly metrics (design): parity vs tolerance by node type · unintended deployments per hundred changes · validation-fail rate · rollbacks by cause · share self-served · legacy consumers without a destination.
TRADE-OFF I WOULD MAKE. Waves by node type over a big-bang switch: running two engines in parallel costs maintenance and dual-run time, but it bounds the blast radius of any rule I translated wrongly to one node type. **FAILURE / LEARNING (A4, record):** at Flexport I set the canonical-source rule after the first executive comparison; once dashboards have constituencies, a cheap definition becomes an expensive negotiation. Here I would publish the parity metric before anyone sees a shadow-mode number.

SAFE-SAY. "I haven't migrated a placement rules engine; this is how I would sequence it, and every step is a mechanism I have run — the 44-row register at Sephora, the three-mock migration and the Flexport decommission." **Source:** "The UDC architecture-council deck, the Canada discovery deck, my resume line on the dual-DC migration and the Crux transition RAPID." **LPs:** Think Big · Insist on the Highest Standards · Dive Deep · Ownership.